

UCS2603 - Theory Of Computation - Assignment 2

Name: A Ankitha Reddy

Reg no: 3122225001013

Batch: 2022-26

Section: 31CSE-A

- 1) Design a Turing machine in inventory management for a warehouse system (unary notation). When a product is sold, the quantity is updated. The TM can perform subtraction between the current stock (m) and quantity sold (n), ensuring that the stock is not negative. If the quantity exceeds the stock, the TM will replace the result with a blank symbol (B) indicating zero stock.

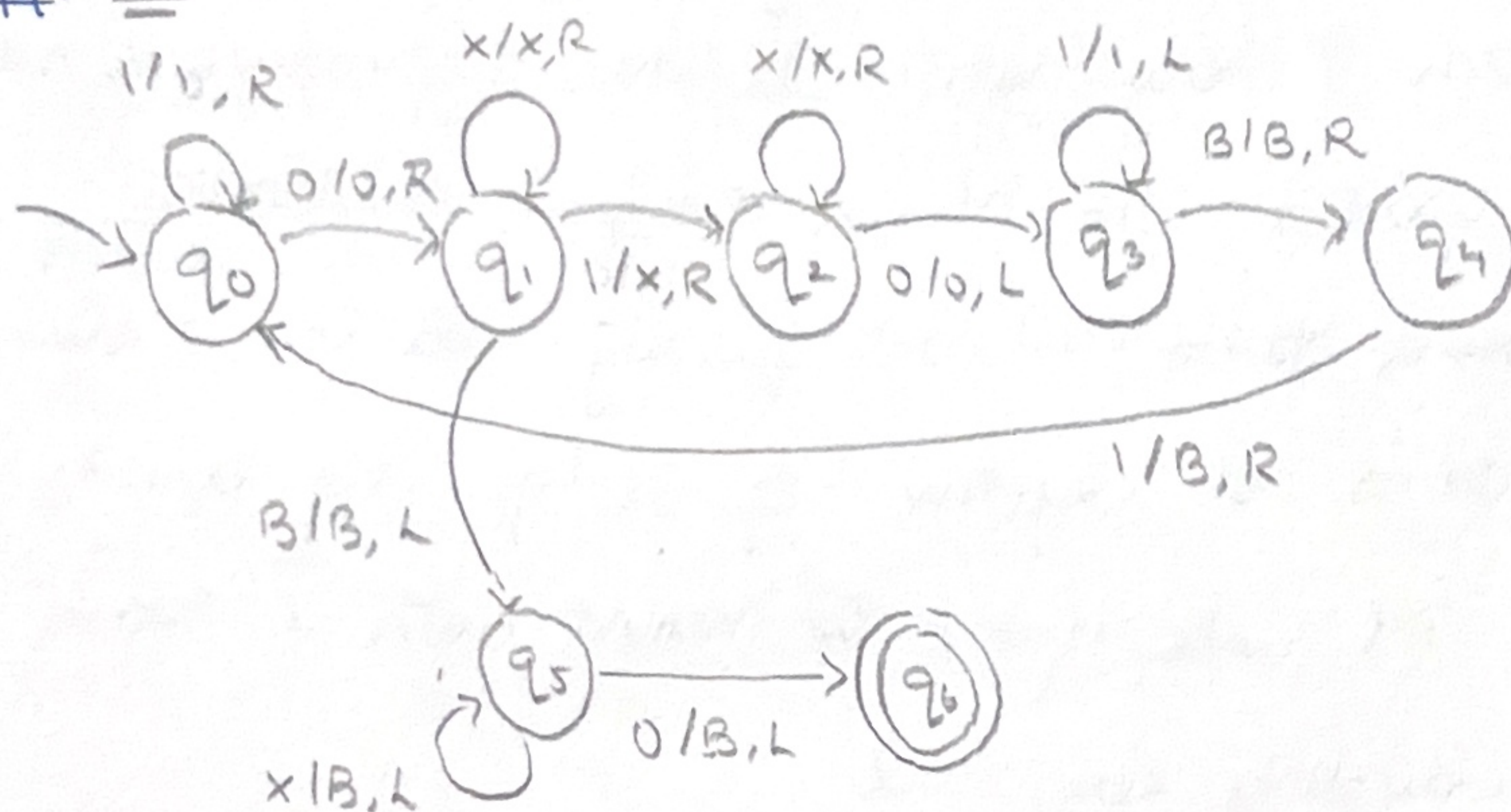
Ex: $B \ 1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ B$
 ↑ ↑
 current quantity of stock sold (n)
 inventory (m)

Steps:

- 1) If 1 is found, convert all 1's into 1's and go right, then convert 0 into 0 and go right.
- 2) If x is found, convert all x into x and go right or if 1 is found then convert 1 to x and go left and go to next step, otherwise step 5.
- 3) Convert all x into x and go left then 0 into 0 and go left.

- 4) Then convert all 1's into 1's and go left and convert B into B and go right and then convert 1 into B and go right and repeat the whole process.
- 5) Otherwise if B is found, convert B into B and go left, then convert c into B and go left and then stop machine.

~~FSM~~: TM:



Here, the problem is a case of simple way subtraction given by:

$$\text{sub}(m, n) = \begin{cases} m - n & : \text{if } m > n \\ 0 & : \text{otherwise} \end{cases}$$

Here, this the Turing machine that can be used as an inventory management system.

- 2) Design a 3-track Turing Machine to solve the following problem. Given a number 'n' in unary, the task is to determine if 'n' is a prime number. A prime no. has no divisors other than 1 and itself (not divisible by any no. between 2 and n-1).

Algorithm:

- 1) Place the input N in track 1
- 2) Set Track 2 = 2.
- 3) Track 3 = (Track 1) % (Track 2)
- 4) Track 2 = (Track 2) + 1
- 5) If Track 3 == 0, deduce 'Not Prime' and go to step 8.
- 6) If Track 2 $\leq$ Track 1 - 1, go to step 3.
- 7) Declare 'Prime'
- 8) Halt

For ex: for $N=5$,

T1	5	5	5
T2	2	3	4
T3	1	2	1

$\Rightarrow 5$ is Prime

In step 3, we have used $\text{Track 3} = \text{Track 1} \% \text{Track 2}$.

Consider the following Turing machine to find $x \bmod y$ for any 2 unary numbers x and y .

Input: x, y

Output: $x \bmod y$

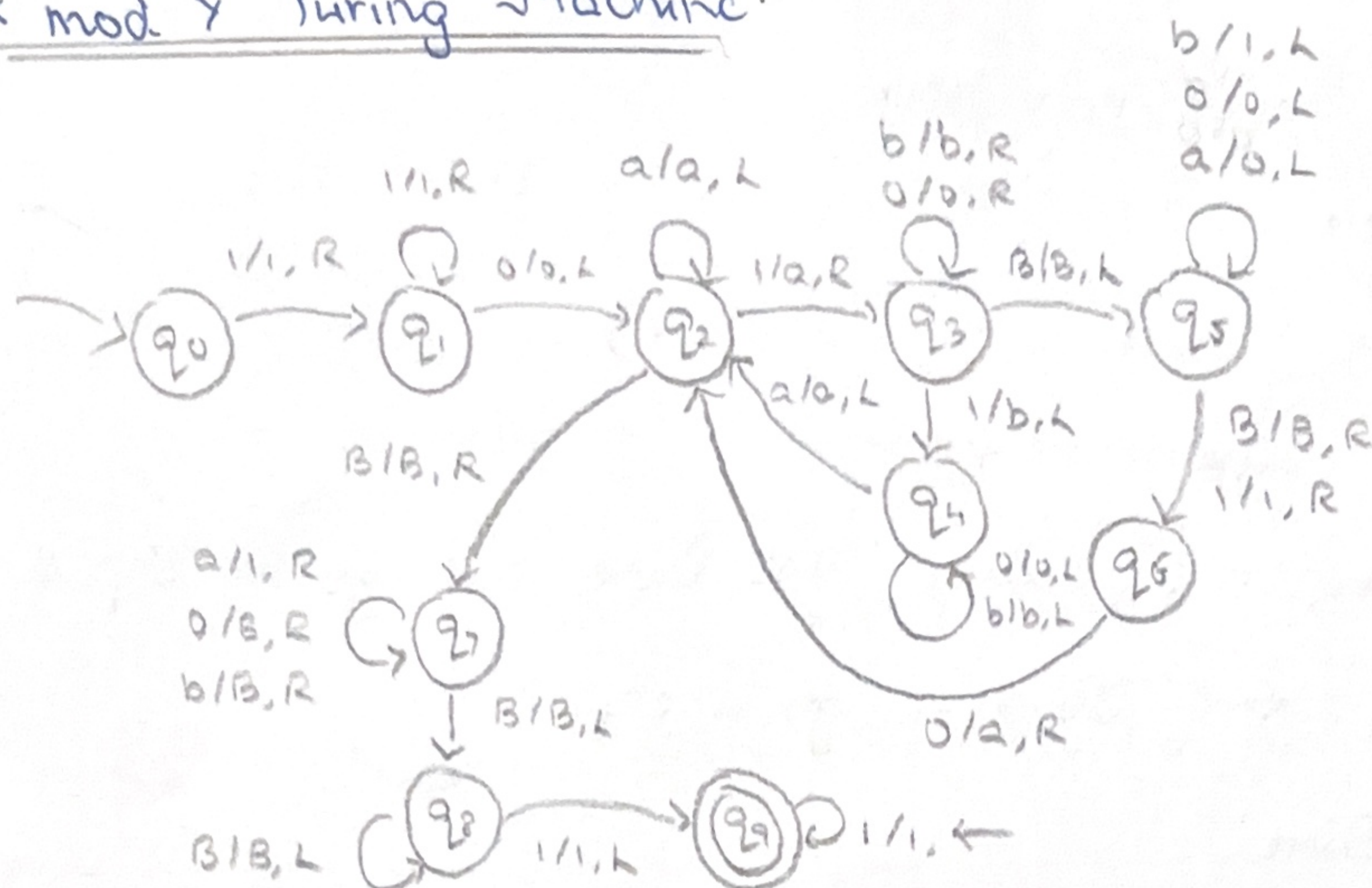
Ex:

Input: 11111011 [5 mod 2]

Output: 1

(P.T.O)

$x \bmod y$ Turing Machine:



The input is 2 positive integers x and y in unary separated by a separator symbol. Output is a single no. $x \bmod y$.

Algorithm explanation:

First move right to find the separator. Then move back and forth between the end of x and the beginning of y , making marking pairs of symbols. If you run out of x before y , then $x < y$ and $x \bmod y = x$. Hence, erase all the symbols after (including the separator), change all the tape symbols to unary and then halt-accept. If you run out of y before x , change the marked symbols in x to separator, restore marked symbols of y to the unary digit, and repeat ($x > y$ means $x \bmod y = (x - y) \bmod y$).

For ex: $2 \bmod 3$

$\Rightarrow B110111B$

$\Rightarrow B1a0b11B$

$\Rightarrow Ba a0bb1B$

$\Rightarrow Ba aBBB BB$

$\Rightarrow B11BBB BB$

$3 \bmod 2$

$\Rightarrow B111011B$

$\Rightarrow B11a0b1B$

$\Rightarrow B1a a0bbB$

$\Rightarrow B100011B$

$\Rightarrow Ba000b1B$

$\Rightarrow BaBBB BB$

$\Rightarrow B1BBB BB$

Hence, a 3-track Turing machine to check for primes is const.
ucted.

3) Explain the distinction between a language that is recursive (R), recursively enumerable (RE), not recursively enumerable and a language that is undecidable but RE. Provide an example of each and discuss their real-world applications.

A language is a set of strings over a given alphabet. In the context of Turing machines (TM), different classifications of languages help us understand the limits of computation and decidability.

i) Recursive Languages (Decidable):

- A language L is recursive if there exists a Turing Machine (TM) M that always halts (either accepts or rejects) for every input w .
- Example: $L = \{w \mid w \text{ is a valid palindrome in binary}\}$

- Real world application: Lexical analysis in compilers - checking if an identifier follows a pre-defined grammar.

ii) Recursively Enumerable Languages (semi-decidable):

- A language L is recursively enumerable if there exists a Turing Machine (T.M) M that halts every string and accepts every string in L , but for some strings not in L , it may loop indefinitely, i.e.
 - if $w \in L$, then T.M will eventually accept
 - if $w \notin L$, then T.M may reject or loop forever
- Example: The Halting Problem Language

$$L = \{ \langle M, w \rangle \mid \text{TM } M \text{ halts on input } w \}$$
- Real world application: Bug detection in software - static analysis tools try to determine if a program will crash but may sometimes run indefinitely.

iii) Not Recursively Enumerable Languages:

- A language L is not R.E if there is no Turing Machine that can even enumerate its members. This means L cannot be recognised by any algorithm.
- Example: Complement of the Halting Problem

$$L = \{ \langle M, w \rangle \mid \text{TM } M \text{ does not halt on input } w \}$$

- Real world application: Proving absolute security. Some cryptographic proofs require proving no attack exists, which is equivalent to non-RE problems.

iv) Undecidable but Recursively Enumerable Languages:

- A language L is undecidable but R.E if:
 - There exists a T.M that recognises L .
 - There is no algorithm that decides L .
 - It is strictly harder than recursive languages but not as hard as not recursively enumerable languages.
- Example: Variation of Halting Problem

$$L = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$$
- Real-world application: AI model explainability as we can decide when the machine stops, but we cannot decide the halting problem (example above). This can be applied here as some AI interpretability questions are undecidable but may be approximated using heuristics.

4) Two versions of files / database need to be verified for consistent states. List A and B give sequences of operations applied to the first and second versions

of data. Choose a technique to find all possible solutions for these lists, it indicates that there is a sequence of operations that can bring both versions to some state.

LA = 10, 101, 011, 0, 110

LB = 101, 011, 1, 01, 110

1) 10 101

2) 101 011

3) 011 1

4) 0 01

5) 110 110

The technique used to find all possible solutions is the so that sequence of operation bring both versions of data to the same state is a mapping to the post correspondence problem (PCP). It contains 2 lists of strings in which we must check whether there exists an index sequence $i_1, i_2, \dots, i_k$ such that

$$A[i_1]A[i_2] \dots A[i_k] = B[i_1]B[i_2] \dots B[i_k]$$

i.e. when we concatenate elements in some order, the results must be identical.

As PCP is an undecidable problem, there is no algorithm to determine if the system has solution or not.

Hence, it is not possible to generate all possible strings through PCP but many can be generated from this method. It follows the assumption that all terms

are not part of the PCP atleast once [If the definition of PCP is such that all terms be there atleast once, then problem has ~~not~~ no solution].

∴ Steps include: Concatenation construction and equality verification

index i_1	$A[1] = 10$	, $B[1] = 101$	[no match]
i_2	$A[2] = 101$	, $B[2] = 01$	[no match]
i_3	$A[3] = 011$	, $B[3] = 1$	[no match]
i_4	$A[4] = 0$	, $B[4] = 01$	[no match]
i_5	$A[5] = 110$	, $B[5] = 110$	[match - solution]

Solutions [not exhaustive]:

$$1) i_1 = 1, i_2 = 2 \rightarrow \left. \begin{array}{l} A[1] + A[2] = 10 + 101 = 10101 \\ B[1] + B[2] = 101 + 01 = 10101 \end{array} \right\} \text{match}$$

$$2) i_1 = 1, i_2 = 2, i_3 = 5 \rightarrow \left. \begin{array}{l} A[1] + A[2] + A[5] = 10101110 \\ B[1] + B[2] + B[5] = 10101110 \end{array} \right\}$$

$$3) i_1 = 4, i_2 = 2 \rightarrow \left. \begin{array}{l} A[4] + A[2] = 0 + 101 = 0101 \\ B[4] + B[2] = 01 + 01 = 0101 \end{array} \right\}$$

$$4) i_1 = 4, i_2 = 2, i_3 = 5 \rightarrow \left. \begin{array}{l} A[4] + A[2] + A[5] = 0101110 \\ B[4] + B[2] + B[5] = 0101110 \end{array} \right\}$$

$$5) i_1 = 5, i_2 = 1, i_3 = 2 \rightarrow \left. \begin{array}{l} A[5] + A[1] + A[2] = 110 + 10 + 101 \\ B[5] + B[1] + B[2] = 110 + 101 + 01 \end{array} \right\}$$

$$6) i_1 = 1, i_2 = 2, i_3 = 5, i_4 = 4, i_5 = 2 \rightarrow$$

$$A[1] + A[2] + A[5] + A[4] + A[2] = 10 + 101 + 110 + 0 + 101$$

$$B[1] + B[2] + B[5] + B[4] + B[2] = 101 + 01 + 110 + 01 + 01 \\ = 101011100101$$